

HAQ — BUILD PRD

Implementation plan for a 36-hour national hackathon

Team	Hack_Overflow (4)
Theme	AI Innovation with Sarvam AI
Deliverable	One live demo, one repo, one 3-min video
Governing rule	If it is not on stage, it does not get built.

PART 0 — How hackathons are actually won

Before any code, internalise this. Judges at national level see 40–200 teams in a day. They spend 5–8 minutes with you. They are tired.

What actually decides the result:

Weight	Factor	What it means for us
30%	The demo works, live, without excuses	Offline demo mode. Non-negotiable. See §7.
25%	Problem legibility in the first 30 seconds	The 90-day cliff. One sentence, no slides.
20%	One unforgettable moment	The refusal. The AI says no.
15%	Visible technical depth	The Ladder YAML on screen + a passing test suite
10%	Q&A composure	Rehearsed answers to 12 hard questions (§9)

What does NOT decide it: feature count, number of screens, login systems, database design, mobile responsiveness, a beautiful landing page.

Three rules that follow:

1. **Build backwards from the 6-minute demo script.** Write the script first (§8). Anything it doesn't touch is cut.
2. **Fake everything that isn't the thesis.** Auth, payments, user management, multi-tenancy — all hardcoded. Nobody will ask.
3. **Depth in exactly one place.** Ours is the Ladder Engine. Everything else is scaffolding around it.

PART 1 — Final scope

1.1 What we build (the whole list)

#	Component	Why it's in
C1	Voice intake — record → Saaras v3 → Marathi/Hindi transcript	Proves Sarvam voice. Opens the demo.
C2	Interrogation agent — Sarvam-105B, one question per turn, fills a fact sheet	Proves agentic behaviour
C3	Ladder Engine — pure function, 2 ladders, unit-tested	The technical moat. The refusal lives here.
C4	Document generator — Mayura → formal English → PDF	The tangible artifact judges can hold
C5	Bulbul read-back — English draft spoken in Marathi	Proves the asymmetric-language thesis
C6	Time-travel daemon — advance clock, escalation fires	The closing wow
C7	Web UI — the existing prototype HTML, wired to the API	Zero build step, already designed
C8	WhatsApp channel — Twilio Sandbox	Optional. Cut first if behind.

1.2 What we explicitly do NOT build

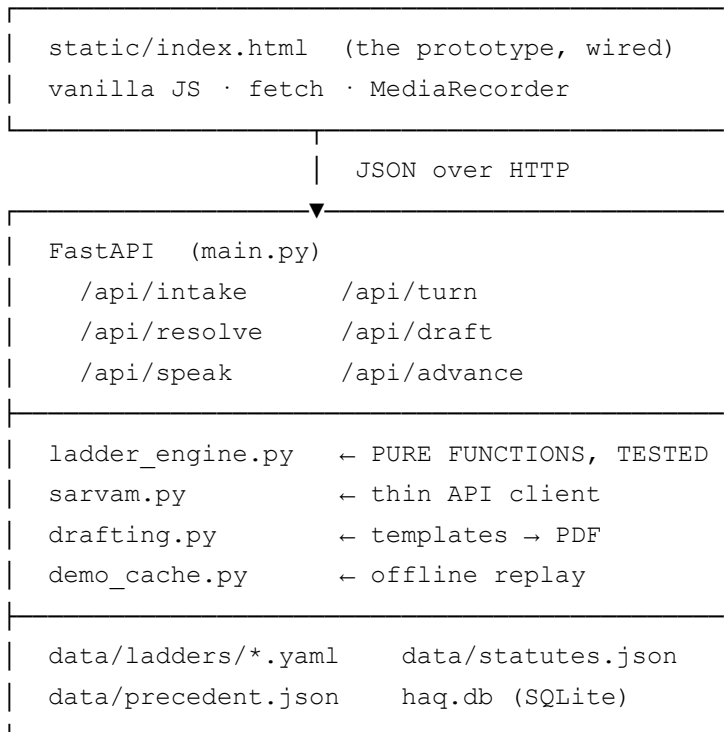
Auth · user accounts · database migrations · Docker · CI/CD · a real RAG pipeline · Sarvam Vision OCR (P1) · Precedent Memory (fake it with a hardcoded JSON) · Haq Score model

(rule-based formula) · operator console (screenshot only) · mobile app · payment · admin panel · Next.js.

The Next.js decision. Kill it. The prototype HTML you already have becomes the production frontend — vanilla JS `fetch` against FastAPI, served as a static file. This saves 6–8 hours of build tooling and dependency hell, and the design is already done. This single decision is the biggest schedule win available to you.

PART 2 — Architecture

2.1 The whole system



Stack: Python 3.11, FastAPI, SQLite (via plain `sqlite3`, no ORM), PyYAML, `fpdf2`, `httpx`.
Frontend: one HTML file, no framework, no npm.

That is the entire dependency list. Nothing else gets installed without a team vote.

2.2 File tree — create this exactly, hour 1

```
haq/
├── main.py                # FastAPI app + all routes (~250 lines)
├── ladder_engine.py       # pure functions (~180 lines) ★ the moat
```

```

├─ sarvam.py                # 5 functions, one per API    (~120 lines)
├─ agent.py                 # 105B loop + prompts        (~150 lines)
├─ drafting.py              # templates → text → PDF    (~120 lines)
├─ demo_cache.py            # offline replay            (~40 lines)
├─ store.py                 # SQLite helpers            (~80 lines)
├─ test_ladder.py           # ★ the credibility artifact (~120 lines)
├─ data/
│   └─ ladders/
│       └─ rbi_rbios_2026.yaml
│       └─ rti_2005.yaml
│   └─ statutes.json        # ~18 hardcoded snippets + source URLs
│   └─ precedent.json       # hardcoded outcome stats
│   └─ cache/               # recorded Sarvam responses for demo mode
├─ static/
│   └─ index.html           # the prototype, wired
│   └─ fonts/DejaVuSans.ttf
├─ .env.example
└─ README.md

```

Total target: under 1,100 lines of Python. If you're writing more, you're building something we cut.

2.3 API surface — six endpoints, that's all

```

POST /api/intake
  body: multipart audio blob, lang hint
  → { case_id, transcript, language, classification, confidence }

POST /api/turn
  body: { case_id, text | audio }
  → { question, question_native, facts_known, facts_pending, done }

POST /api/resolve
  body: { case_id }
  → { ladder_id, current_tier, maintainable, blocked_by,
      redirect_to, deadlines[], haq_score, citations[] }

POST /api/draft
  body: { case_id, tier }
  → { body_text, citations[], pdf_url, grounded: bool }

POST /api/speak
  body: { text, lang }
  → { audio_base64 }

POST /api/advance

```

```
body: { case_id, days }          # the time machine
→ { events[], new_messages[], new_deadlines[] }
```

PART 3 — The Ladder Engine (build this first, it is the product)

3.1 Why it goes first

Everything else is a wrapper. If the engine works and the UI is ugly, you still have a product. If the UI is beautiful and the engine is a prompt, you have a chatbot, and a judge will find that out in Q&A.

3.2 The pure function

```
# ladder_engine.py

@dataclass
class Facts:
    grievance_class: str
    tier1_written_complaint: bool | None
    tier1_filed_on: date | None
    last_communication_on: date | None
    pending_in_court: bool
    amount_inr: float
    account_type: str | None

@dataclass
class Verdict:
    ladder_id: str
    current_tier: int
    maintainable: bool
    blocked_by: list[str]          # exclusion ids
    redirect_to: int | None
    deadlines: list[Deadline]
    next_action: str
    citations: list[Citation]

def resolve(facts: Facts, ladders: dict, today: date) -> Verdict:
    ...
```

Hard requirements:

- No network calls. No LLM. No randomness. Same input → same output, always.

- Every `Verdict` carries the `source_url` and `verified_on` of the ladder it used.
- `blocked_by` is never empty when `maintainable` is `False`, and each entry maps to a plain-language message from the YAML.

3.3 The tests that win Q&A

`test_ladder.py` — twelve cases. Run it on stage if challenged.

```
def test_tier1_not_exhausted_blocks_ombudsman():
    v = resolve(Facts(grievance_class="banking/unauthorised_charge",
                     tier1_written_complaint=False, ...), L, date(2026,8,8))
    assert v.maintainable is False
    assert "tier1_not_exhausted" in v.blocked_by
    assert v.redirect_to == 1

def test_ninety_day_window_computed_from_later_of_two_dates(): ...
def test_time_barred_after_ninety_one_days(): ...
def test_pending_in_court_hard_stops(): ...
def test_commercial_decision_excluded(): ...
def test_deadline_is_deterministic_across_runs(): ...
```

Judge moment: when asked “how do you know the AI isn’t hallucinating the law?” — you run `pytest -v` on the projector. Twelve green dots. That answer beats any explanation.

3.4 Ladder YAML — ship exactly two

`rbi_rbios_2026.yaml` — the headline. Full spec is in the product PRD; port it verbatim.

`rti_2005.yaml` — a second, structurally different ladder. Its only job is to prove the engine is general and not hardcoded to one law. Four fields, thirty minutes of work.

Do not add a third. Breadth reads as guessing; depth reads as rigour.

3.5 statutes.json — skip RAG entirely

Building a retrieval pipeline over Indian statutes is a two-day project. Don’t.

```
[
  {
    "id": "rbi_bsbda_no_amc",
    "keywords": ["bsbda", "jan dhan", "annual maintenance", "amc", "zero balance"],
    "text": "RBI's Master Direction on Basic Savings Bank Deposit Accounts does n
    "source_url": "https://rbi.org.in/...",
    "verified_on": "2026-08-08",
    "verified_by": "Naman"
```

```
}  
]
```

Retrieval = keyword overlap scoring. Eighteen entries, ten lines of code, zero embeddings, zero vector DB. It is *more* accurate than RAG at this scale and infinitely more debuggable.

The grounding validator — this is the line that makes the whole safety story real:

```
def validate_citations(body: str, cited: list[Citation]) -> tuple[str, list[str]]  
    """Strip any citation lacking a source_url retrieved this session."""  
    stripped = []  
    for c in cited:  
        if not c.source_url or c.id not in session_retrieved_ids:  
            body = body.replace(c.text, "")  
            stripped.append(c.id)  
    return body, stripped
```

Say on stage: *“This is code, not a prompt. The model literally cannot emit an ungrounded section number, because the renderer deletes it.”*

PART 4 — Sarvam integration

4.1 Endpoints

Base URL `https://api.sarvam.ai` . Auth header: `api-subscription-key` .

Function	Model	Notes
<code>stt(audio, lang)</code>	<code>saaras:v3</code>	Use <code>mode="codemix"</code> . Multipart upload.
<code>chat(messages, tools)</code>	<code>sarvam-105b</code>	OpenAI-compatible chat completions
<code>translate(text, tgt)</code>	Mayura / <code>sarvam-translate</code>	<code>mode="formal"</code> for filings
<code>tts(text, lang)</code>	<code>bulbul:v3</code>	Returns base64 wav; chunk to 500 chars
<code>parse(file)</code>	Sarvam Vision	P1 — cut from hackathon scope

Get the exact request paths from `docs.sarvam.ai`. Their docs expose an agent-readable index — append `/llms.txt` to any docs URL for a page index, or `.md` for the markdown of any page. Feed that straight to your coding assistant instead of guessing schemas. This will save an hour.

4.2 The audio trap — read this before writing any recorder code

Browser `MediaRecorder` outputs **webm/opus**. Sarvam STT expects **wav/mp3**. Every team hits this at 2am.

Two fixes, in order of preference:

1. **Encode WAV in the browser.** Use a small AudioWorklet/ScriptProcessor PCM→WAV encoder (~60 lines, copy a known-good gist). No server dependency. **Do this.**
2. Server-side `ffmpeg` conversion. Works, but adds a system dependency that will break on someone's machine at hour 30.

Test this in **hour 3**, not hour 25. It is the single most likely cause of a dead demo.

4.3 The agent loop — keep it dumb

Do not build a general tool-calling harness. Build a state machine with an LLM inside it:

```
STATE: COLLECTING
  → 105B gets (fact_sheet, ladder_requirements) and returns ONE question
  → answer parsed into the fact sheet
  → repeat until all required facts present [max 6 turns, hard cap]

STATE: RESOLVING
  → ladder_engine.resolve() ← no LLM involved
  → if blocked: 105B phrases the refusal in the user's language

STATE: DRAFTING
  → template + retrieved statutes + case facts → 105B fills the body
  → validate_citations() strips anything ungrounded
  → Mayura → formal English → fpdf2 → PDF

STATE: WATCHING
  → deadlines in SQLite; /api/advance simulates the clock
```

Four states. No autonomous planning. Deterministic where it matters, generative only where language is the output. This is also the correct architecture, not just the fast one — and you should say so when asked.

PART 5 — 36-hour schedule

Four people. Named owners. Definition of done at each checkpoint.

Hours 0–2 · Everyone — Lock the demo

Do not open an editor. Write the 6-minute demo script (§8) on a whiteboard, agree it, and photograph it. Then assign owners and create the file tree.

DoD: demo script agreed, repo scaffolded, `.env` shared, everyone can run `uvicorn`
`main:app`.

Hours 2–10

Owner	Task	DoD
Krishnan (lead)	<code>ladder_engine.py</code> + <code>rbi_rbios_2026.yaml</code> + <code>test_ladder.py</code>	<code>pytest</code> green, 12 tests, refusal case passing
Yash	<code>sarvam.py</code> : stt, tts, chat, translate. WAV encoder in browser.	Speak Marathi into the browser → correct transcript in terminal
Ayush	Wire prototype HTML to a live API; <code>store.py</code> + SQLite	Record button → POST → transcript renders on screen
Naman	<code>statutes.json</code> (18 entries), verify RBI clauses, <code>precedent.json</code>	Every entry has a working <code>source_url</code> and a name in <code>verified_by</code>

Checkpoint at hour 10: voice in → transcript out → engine returns a verdict. Ugly is fine.

Hours 10–20

Owner	Task	DoD
Krishnan	<code>agent.py</code> state machine + interrogation prompt + <code>rti_2005.yaml</code>	5-turn conversation fills the fact sheet reliably 3x in a row
Yash	<code>drafting.py</code> : template → 105B →	A real PDF downloads and reads correctly

	Mayura → fpdf2; grounding validator	
Ayush	Case detail screen live: ladder stepper, refusal banner, deadlines	Refusal renders with the exclusion id and source link visible
Naman	/api/advance + deadline logic + demo_cache.py recorder	Advancing 31 days produces the escalation message

Checkpoint at hour 20 — FEATURE FREEZE. Whatever exists now is the product. From here, only polish, caching, and rehearsal.

Hours 20–28

Owner	Task
Krishnan	Owens the demo from here. Runs it end to end 10 times. Logs every break.
Yash	Record every Sarvam response into <code>data/cache/</code> . Build <code>DEMO_MODE=1</code> replay.
Ayush	Fix only what Krishnan's runs break. No new UI.
Naman	Deck (8 slides), 3-min video, README, fact-check every number

Hours 28–34

- **Hour 28:** full dress rehearsal, timed, on the actual demo laptop, on hotspot.
- **Hour 30:** run the entire demo with **wifi switched off**. If it fails, you have 6 hours to fix it. This is the most valuable hour of the whole event.
- **Hour 32:** Q&A drill — Naman fires all 12 questions from §9, everyone answers cold.
- **Hour 34:** freeze. Commit. Push. Stop.

Hours 34–36 — Sleep. Seriously.

A rested team answering Q&A well beats an exhausted team with one more feature. Every time.

PART 6 — The cut ladder

When you fall behind — you will — cut in this exact order. Decide by vote, in under 60 seconds, no debate.

Cut #	Drop this	Recover
1	WhatsApp / Twilio channel	4 hrs
2	The second ladder (RTI)	1.5 hrs
3	PDF generation → styled HTML letter instead	2 hrs
4	Live voice input → pre-recorded audio file upload	3 hrs
5	Mayura translation step → draft directly in English	1 hr
6	Bulbul read-back → show text only	1.5 hrs

Never cuttable: the Ladder Engine, the refusal, the time-travel escalation, `test_ladder.py`. Those four *are* the submission.

PART 7 — Demo mode (the most important section)

Rule: never demo live against a third-party API on venue wifi. Venue wifi fails. Sarvam rate-limits. A cold model call takes 9 seconds and the room goes quiet.

Implementation — 40 lines

```
# demo_cache.py
CACHE = Path("data/cache")
MODE = os.getenv("HAQ_MODE", "live")      # live | record | replay

def cached(key: str, fn):
    f = CACHE / f"{key}.json"
    if MODE == "replay":
        time.sleep(0.4)                  # keep it feeling real, not instant
        return json.loads(f.read_text())
    out = fn()
    if MODE == "record":
        f.write_text(json.dumps(out))
    return out
```

Wrap every Sarvam call. Key on a hash of the input.

Demo day protocol:

- `HAQ_MODE=replay` on the presenting laptop. Always.
- A second laptop runs `HAQ_MODE=live`, on a phone hotspot, as the “yes it’s real” backup.
- If a judge says “*is this hardcoded?*” — you switch to the live laptop and do one call in front of them. **Volunteering this proactively is stronger than being asked.** Say: “*The demo runs from cache so it never fails on venue wifi. Here’s the same call live — watch.*” That converts a suspicion into a credibility win.
- Never say the word “hardcoded” defensively. Say “cached, and here’s the live one.”

PART 8 — The demo script

Six minutes. Rehearsed 10 times. Krishnan presents; Ayush drives the laptop.

0:00 — Hook, no slide (40s)

“On the first of July this year, the RBI quietly cut the time you have to escalate a bank complaint — from one year to ninety days. Nobody told you. Nobody told the two hundred million people it affects. We built the alarm clock.”

0:40 — Sunita speaks, live (50s) Speak a rambling Marathi grievance into the laptop. Transcript appears. Classification appears. No forms touched. “*That’s the entire onboarding.*”

1:30 — Interrogation (50s) Three questions, one at a time. The fact sheet fills. Land the point: “*It’s asking the two questions that legally control everything — was the bank complained to in writing, and when did they last respond.*”

2:20 — ★ THE REFUSAL (70s) — the moment you win Screen shows `MAINTAINABILITY: BLOCKED · Clause 10 · tier1_not_exhausted`, with the RBI source link visible.

“Every other AI product would have cheerfully generated an Ombudsman complaint here. It would have been rejected, and she’d have burned her only chance. Ours knows the rules well enough to say no — and that’s not a prompt, it’s a tested function.” Alt-tab. `pytest -v`. Twelve green dots. Alt-tab back. **Do not explain the tests. Let them land.**

3:30 — The artifact (60s) Correct tier-1 letter renders in formal English. Bulbul reads it aloud in Marathi. Say “हो.” Filed. Two dated deadlines appear.

“She speaks Marathi. The bank receives formal English. She approved a document she

cannot read, by hearing it in a language she can."

4:30 — Time travel (50s) Click "advance 31 days." The daemon fires unprompted. Marathi voice note arrives. Tier-2 draft is pre-built with the 90-day window shown.

"Nobody clicked anything. Silence became a legal event."

5:20 — The moat (40s) Ladder YAML on screen.

"Two ladders today. There are hundreds. Each one is data, version-controlled, with a source URL and a verification date — so when RBI changes the scheme, it's a pull request, not a retrain."

6:00 — Close (one line, then stop talking)

"India has a thousand AIs that explain the system. This one stands inside it, next to you."

PART 9 — Q&A drill

Twelve questions. Every member must answer all twelve cold. Answers under 25 seconds.

1. **"How do you stop it hallucinating law?"** → Two layers: the ladder is a tested pure function with no LLM in it, and the renderer deletes any citation without a source URL retrieved that session. Show `validate_citations`.
2. **"Is this legal advice? Are you practising law?"** → No. Document preparation and deadline tracking. The user files; we prepare. High-value cases hand off to advocates.
3. **"What if you're wrong and it hurts someone?"** → We never file autonomously. Every outbound document needs a recorded human approval, and we read it aloud first.
4. **"What happens when the law changes?"** → It already did — RB-IOs 2021 became 2026 six weeks ago. That's why ladders are versioned YAML with `effective_from` and `verified_on`, not code.
5. **"Why Sarvam and not GPT/Gemini?"** → Code-mixed Marathi over a phone mic, native numerals, 8kHz telephony audio, Devanagari handwriting. Our entire input distribution is the thing Sarvam is built for and others aren't.
6. **"Why won't a bank just ignore an AI-drafted complaint?"** → It's indistinguishable from a good human draft, and the ladder makes ignoring it expensive — silence auto-unlocks the Ombudsman, where the entity has to answer the regulator.
7. **"How do you make money?"** → Free for individuals forever. Revenue from lawyer referrals, CSC operator subscriptions, union licensing, and aggregate data for

regulators.

8. **"How do you scale to hundreds of ladders?"** → Each is a few hours of a paralegal's time producing YAML. It's an editorial pipeline, not an engineering one. That's deliberate.
 9. **"What's your moat? Anyone can copy this."** → The ladders take a month to copy. The outcome corpus — what actually works against which institution at which tier — compounds with every case and can't be.
 10. **"How do people find you?"** → Not app stores. CSC operators, ASHA workers, gig unions, tenant associations. One operator serves hundreds.
 11. **"What if someone weaponises this to harass?"** → Respondents must be institutions, enforced at the schema level. You cannot open a case against a private individual.
 12. **"What's not working yet?"** → Answer honestly and specifically. Vision OCR isn't wired, we have two ladders not six, and precedent data is seeded. **Judges trust teams that name their gaps.** Never say "nothing."
-

PART 10 — Submission checklist

Do these at hour 33, not hour 36.

- **Rotate the five leaked keys** in the old repo (Groq × 2, Sarvam, Mistral, ngrok) and purge git history. A judge who finds a live key in your repo will remember it.
- `.env.example` committed, `.env` gitignored
- README: one-paragraph problem, one GIF of the refusal, 3-command setup, architecture diagram, honest "what's not built yet"
- `pytest` passes on a clean clone
- 3-min video: same script as §8, screen recording, no talking-head intro
- Deck: 8 slides max — problem / the 90-day cliff / demo / the refusal / architecture / Sarvam usage / moat / team
- Every statistic has a source URL in the speaker notes
- Demo laptop charged, `HAQ_MODE=replay`, browser zoom at 125%, notifications off, one tab
- Backup laptop with `HAQ_MODE=live` on hotspot
- PDF of a generated complaint printed on paper — hand it to a judge. Physical artifacts

are disproportionately memorable.

PART 11 — Top risks

Risk	When it bites	Mitigation
Browser audio format mismatch	Hour 25	Test in hour 3 ; browser-side WAV encoder
Venue wifi / API failure	On stage	<code>HAQ_MODE=replay</code> , always
105B returns malformed JSON	Hours 12–20	Retry once, then rule-based fallback. Never let a parse error surface.
Scope creep past hour 20	Hours 20–30	Written feature freeze. Lead has veto.
Devanagari renders as boxes in the PDF	Hour 26	Filings are English-only; bundle DejaVuSans for fpdf2
A judge finds a wrong legal claim	Q&A	Every statute entry verified, initialled, source-linked
Team exhaustion	Hour 30+	Mandatory sleep 34–36. Enforced by the lead.

The one-sentence version

Build the Ladder Engine and its tests in the first ten hours, wire the prototype HTML you already have to six endpoints, cache every Sarvam call so the demo cannot fail, and spend the last six hours rehearsing the moment where the AI refuses to file.

Everything else is optional.